



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

GRADO EN INGENIERÍA INFORMÁTICA

PROYECTO COMPUTACIÓN II

TrafiVision

Grupo 5

Anyi Lin

Rodrigo DÍAZ MARTÍNEZ

Ángel FERNÁNDEZ-LUQUE MAGARIÑO

Yul Brynner Cardenas Caso

Dirigido por

EMILIO YOSHIHIRO ROSANES NISIAMA

CURSO 2025-2026

TÍTULO: XXXXXX

AUTORES: Rodrigo DÍAZ MARTÍNEZ Ángel FERNÁNDEZ-LUQUE MAGARIÑO Yul Brynner
Cardenas Caso

TITULACIÓN: INGENIERIA INFORMÁTICA

DIRECTOR/ES DEL PROYECTO: MIQUEL COLOBRAN HUGUET

FECHA: FEBRERO 2026

Control de versión de esta plantilla

Este control de versiones de la plantilla se debe eliminar de la memoria de proyecto. Se incluye aquí para asegurar que se utiliza la versión más actualizada de plantilla de memoria PFG, publicada en el campus virtual.

Nombre del fichero de la plantilla	Versión de esta plantilla	Fecha	Comentarios
Anteproyecto	v1.0	febrero/2026	Versión inicial
Documentacion Fase 1		marzo/26	

RESUMEN

El resumen tiene entre 150-250 palabras. Resumir consiste en ofrecer información sobre cómo, dónde, cuándo y por qué se aplica el proyecto. Se realiza al finalizar el trabajo.

Debe incluir:

- Resumen del problema planteado y las aportaciones más importantes del proyecto al respecto
- Indica si procede si el proyecto se realizó en colaboración con una empresa, indicando nombre (siempre que haya autorización expresa por la empresa), y el sector industrial o empresarial
- Principales resultados del proyecto (diseño de un sistema para..., desarrollo e implementación de una solución para..., análisis de..., modelo de...)
- Resume las conclusiones más importantes del trabajo realizado

El resumen NO

- Da una información genérica
- Se refiere a datos aportados en el texto del proyecto.

Palabras clave: hasta un máximo de 6 conceptos (un concepto puede conllevar una o más palabras). Incluye tecnologías que hayas utilizado, conceptos relevantes del ámbito científico-técnico, y conceptos de la industria (algunos ejemplos: machine learning, Big Data, Smart City, visión artificial, social media, Twitter, computación afectiva, transformación digital, GDPR, DevOps, EEG, ...)

ABSTRACT

Resumen en inglés.

Keywords: Palabras clave en inglés

AGRADECIMIENTOS

En ocasiones se incluye este apartado para agradecer a aquellos que han ofrecido su ayuda en el desarrollo del trabajo, ya sea técnica o de otro tipo.

Índice

RESUMEN.....	4
ABSTRACT.....	5
Capítulo 1. RESUMEN DEL PROYECTO.....	10
1.1 Contexto y justificación.....	10
1.2 Planteamiento del problema.....	11
1.3 Objetivos del proyecto.....	12
1.4 Resultados obtenidos.....	13
1.5 Estructura de la memoria.....	13
Capítulo 2. ANTECEDENTES / ESTADO DEL ARTE.....	14
2.1 Estado del arte.....	14
2.2 Planteamiento del problema.....	14
Capítulo 3. OBJETIVOS.....	20
3.1 Objetivos generales.....	20
3.2 Objetivos específicos.....	22
Capítulo 4. DESARROLLO DEL PROYECTO.....	25
4.1 Planificación del proyecto.....	25
4.2 Descripción de la solución, metodologías y herramientas empleadas.....	27
4.3 Recursos requeridos.....	29
4.4 Presupuesto.....	31
4.5 Viabilidad.....	32
4.6 Resultados del proyecto.....	32
Capítulo 5. TECNOLOGIAS USADAS.....	33
5.1 Tecnologías utilizadas en la fase anterior (Proyecto de Computación I).....	33
5.2 Tecnologías incorporadas en la fase actual.....	34
5.3 Evolución tecnológica del proyecto.....	38
Capítulo 6. ANALISIS Y DISEÑO DE LA SOLUCIÓN.....	40
Capítulo 7. IMPLEMENTACIÓN.....	42
Capítulo 8. CONCLUSIONES.....	46
8.1 Conclusiones del trabajo.....	46
8.2 Puntos débiles del trabajo.....	46
8.3 Conclusiones personales.....	46
8.4 Futuras mejoras.....	46
Capítulo 9. GLOSARIO.....	47
Capítulo 10. BIBLOGRAFIA.....	48
Capítulo 11. ANEXOS.....	49

Índice de Figuras

Todas las figuras incluidas en la memoria deben estar referenciadas para incluir en este índice.

Índice de Tablas

Todas las tablas incluidas en la memoria deben estar referenciadas para incluir en este índice.

Capítulo 1. RESUMEN DEL PROYECTO

Este capítulo es un resumen, y no debe incluir el detalle. Existen capítulos específicos para añadir detalle de cada apartado. por tanto, únicamente se hace un resumen del trabajo completo, siguiendo la estructura marcada por las secciones de este capítulo.

Ocupa un máximo de 1 página.

1.1 Contexto y justificación

El análisis de datos abiertos relacionados con el tráfico se ha convertido en una herramienta esencial para **mejorar la movilidad urbana, reducir la congestión y optimizar la planificación del transporte público y privado**. Sin embargo, la información disponible suele proceder de **fuentes heterogéneas**, con distintos formatos, niveles de actualización y estructuras, lo que **complica su integración y análisis conjunto**.

En este contexto, el proyecto **TrafiVison** nace con el propósito de **unificar y automatizar la recopilación de datos de tráfico y clima en la ciudad de Madrid**, creando un flujo ETL (Extracción, Transformación y Carga) que garantice la coherencia y la trazabilidad de la información. El sistema combina imágenes captadas desde cámaras públicas del Ayuntamiento de Madrid con datos meteorológicos en tiempo real de la API **Open-Meteo**, generando un dataset propio, estructurado y limpio, listo para ser utilizado en futuros análisis.

Desde el punto de vista **académico**, TrafiVison representa un **caso práctico de aplicación de técnicas de ingeniería de datos** dentro del ámbito de la inteligencia artificial y el aprendizaje automático, abordando todas las etapas del ciclo de vida de los datos: obtención, limpieza, integración y validación.

En el plano **social y tecnológico**, el proyecto contribuye al **aprovechamiento de los datos abiertos** como herramienta de conocimiento, fomentando la transparencia y la innovación en la gestión de la movilidad urbana. A largo plazo, TrafiVison busca sentar las bases para el desarrollo de **modelos predictivos** que ayuden a anticipar niveles de tráfico y a diseñar estrategias de movilidad más sostenibles y seguras.

1.2 Planteamiento del problema

El proyecto TrafiVision parte de una problemática ya abordada en la fase anterior: la dificultad de integrar y explotar de forma conjunta datos abiertos de tráfico urbano y condiciones meteorológicas para su análisis y predicción. En Proyecto de Computación I se resolvió esta cuestión desde una perspectiva técnica, desarrollando un sistema ETL automatizado capaz de generar un dataset propio y entrenar modelos predictivos de nivel de tráfico.

Sin embargo, aunque el sistema desarrollado demostró la viabilidad técnica del enfoque, su arquitectura estaba orientada a la experimentación local. El modelo predictivo, los scripts ETL y la aplicación gráfica funcionaban de forma integrada en un entorno de desarrollo, sin una separación clara entre capas, sin persistencia estructurada en una base de datos relacional y sin acceso mediante servicios web.

Por tanto, el problema que se aborda en esta nueva fase no es ya la generación del dato ni la validación del modelo, sino la evolución del sistema hacia una arquitectura más escalable, estructurada y cercana a un entorno profesional.

La pregunta motriz que guía esta etapa del proyecto es la siguiente:

¿Cómo transformar un sistema local de análisis y predicción de tráfico en una aplicación web estructurada, con base de datos persistente, backend con endpoints y gestión de usuarios, manteniendo la coherencia del modelo predictivo desarrollado?

Desde el punto de vista científico-técnico, el proyecto se enmarca en el ámbito de la ingeniería del software y la arquitectura de sistemas web, abordando la separación entre frontend y backend, el diseño de bases de datos, la exposición de servicios REST y la gestión de autenticación y roles.

No se trata de resolver un problema empresarial concreto, sino de trasladar una solución académica funcional a un modelo arquitectónico más robusto, modular y desplegable. En este sentido, el componente innovador del proyecto radica en la integración completa de un pipeline de datos y aprendizaje automático dentro de una arquitectura web estructurada,

permitiendo que un sistema inicialmente experimental evolucione hacia una aplicación con características propias de un entorno real.

1.3 Objetivos del proyecto

El objetivo principal de esta nueva fase del proyecto TrafiVision consiste en evolucionar el sistema desarrollado previamente hacia una arquitectura web estructurada, escalable y preparada para un entorno más cercano al profesional.

En la fase anterior se logró automatizar la recopilación de datos de tráfico y meteorología, generar un dataset limpio y entrenar un modelo predictivo capaz de estimar el nivel de tráfico urbano. En esta etapa, el objetivo ya no es la validación del modelo, sino su integración dentro de una arquitectura más sólida que permita su uso a través de una aplicación web.

De forma resumida, los objetivos de esta fase son:

- Diseñar e implementar una base de datos relacional que sustituya el almacenamiento en archivos CSV y permita una gestión estructurada y persistente de la información.
- Desarrollar un backend que exponga endpoints para consultar datos históricos y realizar predicciones de tráfico.
- Implementar una interfaz web que permita interactuar con el sistema desde un navegador, separando claramente frontend y backend.
- Incorporar un sistema de autenticación con gestión de usuarios y roles.
- Preparar la aplicación para su futura puesta en producción mediante una arquitectura modular y organizada.

En conjunto, esta fase del proyecto busca transformar una solución experimental ejecutada en local en una aplicación web completa, estructurada y extensible.

1.4 Resultados obtenidos

Incluye el resumen de los resultados obtenidos al final de tu proyecto. En el capítulo correspondiente a resultados se detallan estos resultados.

1.5 Estructura de la memoria

Describe muy brevemente los capítulos y su contenido, con tus propias palabras.

Capítulo 2. ANTECEDENTES / ESTADO DEL ARTE

2.1 Estado del arte

Una frase inicial que describa de forma general el planteamiento del problema.

Presentar la literatura / estado del arte analizado en relación al problema.

2.2 Planteamiento del problema

Tras el análisis del estado del arte, se observa que existen múltiples iniciativas orientadas a la visualización de tráfico urbano en tiempo real y al uso de datos abiertos en entornos Smart City. Sin embargo, muchas de estas soluciones se limitan a mostrar información fragmentada, sin integrar de forma estructurada la recopilación automática de datos, su tratamiento, su modelado predictivo y su explotación dentro de una arquitectura completa y desplegable.

Situación previamente resuelta

En la fase anterior del proyecto (Proyecto de Computación I), se abordó la problemática inicial relacionada con la integración y explotación de datos abiertos de tráfico y meteorología.

Se desarrolló un sistema capaz de:

- Automatizar la descarga de imágenes de cámaras públicas.
- Integrar datos meteorológicos en tiempo real.
- Aplicar visión por computador para detectar vehículos.
- Generar un dataset limpio y estructurado.

- Entrenar modelos de aprendizaje automático para predecir el nivel de tráfico.
- Implementar una aplicación de escritorio para consultar predicciones.

Con ello, se resolvió el problema técnico de generación del dato y validación del modelo predictivo, demostrando que era posible construir un sistema funcional de análisis y predicción del tráfico urbano a partir de datos abiertos.

Limitaciones detectadas

El sistema desarrollado en la fase anterior presentaba diversas limitaciones estructurales propias de un entorno académico y de un prototipo inicial. Aunque permitió validar técnicamente el enfoque planteado y demostrar la viabilidad del modelo predictivo, su arquitectura no estaba preparada para evolucionar hacia un sistema más robusto o escalable.

En primer lugar, el almacenamiento de los datos se realizaba mediante archivos CSV. Esta solución resultó adecuada durante las primeras etapas del proyecto, ya que permitía trabajar de forma sencilla con los datos y facilitaba las tareas de limpieza, transformación y entrenamiento de modelos. Sin embargo, a medida que el volumen de información aumentaba, esta aproximación comenzó a mostrar sus limitaciones. Los archivos CSV no permiten gestionar de forma eficiente grandes cantidades de datos ni garantizan aspectos importantes como la integridad referencial, la consistencia de la información o el acceso concurrente por parte de distintos procesos.

En segundo lugar, la arquitectura del sistema no estaba claramente separada en componentes diferenciados. La lógica de extracción y procesamiento de datos, el entrenamiento del modelo de aprendizaje automático y la interfaz de usuario se encontraban integrados dentro del mismo entorno de ejecución. Esta estructura funcionaba correctamente para un prototipo local, pero dificultaba la escalabilidad del sistema y la reutilización independiente de los distintos módulos.

Además, el acceso al modelo predictivo estaba limitado a una aplicación ejecutada de forma local. Esto implicaba que el sistema únicamente podía utilizarse desde el equipo en el que se encontraba instalado el proyecto, lo que impedía su uso desde otros dispositivos o su integración con aplicaciones externas.

Otro aspecto relevante es que el sistema no contemplaba ningún mecanismo de gestión de usuarios. La aplicación estaba diseñada para un único usuario y no incorporaba funcionalidades de autenticación, autorización o control de acceso que permitieran gestionar distintos perfiles de uso.

Finalmente, la organización general del proyecto no estaba pensada para un despliegue real ni para un entorno multiusuario. La estructura del código estaba orientada principalmente al desarrollo experimental y a la ejecución local, lo que dificultaba su adaptación a una arquitectura más estructurada basada en servicios, base de datos persistente y acceso remoto.

Estas limitaciones no impedían el funcionamiento del sistema, pero sí evidenciaban la necesidad de evolucionar el proyecto hacia una arquitectura más organizada, modular y preparada para entornos de uso más amplios, objetivo que se aborda en la fase actual del proyecto.

Planteamiento actual del problema

En esta nueva fase del proyecto, el problema ya no se centra en la generación o en la calidad de los datos, ya que estos aspectos fueron abordados en la etapa anterior. En Proyecto de Computación I conseguimos construir un sistema capaz de recopilar, procesar e integrar datos de tráfico y meteorología, así como entrenar modelos de aprendizaje automático para estimar el nivel de tráfico urbano. Esto permitió demostrar la viabilidad técnica del enfoque planteado y validar el uso de datos abiertos para el desarrollo de modelos predictivos.

Sin embargo, durante el análisis del sistema desarrollado detectamos que la principal limitación ya no estaba relacionada con los datos ni con el modelo,

sino con la arquitectura del propio sistema. La solución existente funcionaba correctamente en un entorno local, pero estaba basada principalmente en scripts y archivos planos, lo que dificultaba su evolución hacia una aplicación más estructurada y escalable.

A partir de esta situación surge la necesidad de replantear la arquitectura del sistema. La pregunta que guía esta nueva fase del proyecto puede formularse de la siguiente manera: ¿cómo evolucionar un sistema local de análisis y predicción basado en scripts y archivos CSV hacia una arquitectura más profesional, con persistencia en una base de datos, un backend que exponga servicios mediante endpoints y una interfaz accesible desde navegador, manteniendo al mismo tiempo la coherencia del modelo predictivo ya desarrollado?

Este planteamiento responde principalmente a una necesidad de mejora arquitectónica más que a una falta de conocimiento científico o técnico. El reto consiste en transformar una solución experimental orientada a pruebas locales en un sistema más estructurado, modular y preparado para evolucionar hacia una aplicación web completa.

De esta forma, el proyecto adopta en esta fase una orientación más cercana al desarrollo de producto y a la arquitectura de software, centrándose en la organización del sistema, la persistencia de datos, la separación de componentes y la preparación de la aplicación para futuros escenarios de uso más amplios.

Propuesta de resolución en esta fase

Para dar respuesta a la problemática identificada, en esta nueva fase del proyecto planteamos una evolución progresiva de la arquitectura del sistema. El objetivo no es modificar el modelo predictivo desarrollado anteriormente, sino reorganizar y estructurar el sistema para que pueda integrarse dentro de una aplicación más completa y preparada para futuros escenarios de uso.

En primer lugar, proponemos sustituir el almacenamiento basado en archivos CSV por una base de datos relacional que permita gestionar la información de

forma estructurada y persistente. Esta base de datos permitirá almacenar los datos de tráfico, las capturas de las cámaras, la información meteorológica y las detecciones realizadas por el sistema, garantizando integridad, consistencia y facilidad de consulta.

En segundo lugar, se plantea desarrollar un backend que actúe como capa intermedia entre la base de datos, el modelo predictivo y la interfaz de usuario. Este backend expondrá distintos servicios mediante endpoints, permitiendo consultar información histórica almacenada en la base de datos y ejecutar predicciones utilizando los modelos de aprendizaje automático previamente entrenados.

Otro de los objetivos es separar claramente la lógica de negocio del sistema respecto a la capa de presentación. En la solución inicial, muchas de estas funcionalidades estaban integradas dentro del mismo entorno de ejecución. En esta fase buscamos reorganizar el proyecto siguiendo una estructura modular que permita mantener independientes los componentes relacionados con el acceso a datos, la lógica del sistema y la interfaz de usuario.

Asimismo, se plantea desarrollar una interfaz accesible desde navegador que permita interactuar con el sistema de forma más flexible. Esta interfaz permitirá consultar datos históricos, visualizar información relevante y realizar predicciones de tráfico sin depender de una ejecución local del proyecto.

Además, se prevé incorporar un sistema básico de autenticación y gestión de usuarios que permita diferenciar distintos tipos de acceso dentro de la aplicación, como usuarios estándar o administradores, facilitando así un uso más controlado del sistema.

Finalmente, todas estas mejoras se organizarán siguiendo una arquitectura modular que facilite la evolución futura del proyecto y su posible despliegue en entornos más cercanos a producción.

De este modo, TrafiVision evoluciona progresivamente desde una solución experimental validada técnicamente hacia una aplicación más estructurada y organizada, alineada con prácticas habituales de desarrollo de software y preparada para seguir creciendo en fases posteriores del proyecto.

Capítulo 3. OBJETIVOS

3.1 Objetivos generales

Para alcanzar el objetivo general del proyecto, el trabajo se organizará en tres fases principales, cada una con objetivos técnicos concretos que permitirán evolucionar progresivamente el sistema TrafiVision hacia una aplicación web completa y preparada para producción.

Fase 1: Diseño de arquitectura base (Fecha de entrega: 15/03)

En esta primera fase del proyecto nos centramos en establecer la estructura técnica fundamental del sistema, con el objetivo de comenzar la transición desde el prototipo desarrollado en la fase anterior hacia una arquitectura más organizada y preparada para futuras ampliaciones.

En primer lugar, se diseñó e implementó una base de datos relacional que sustituyera el sistema de almacenamiento basado en archivos CSV utilizado en la fase anterior del proyecto. Para ello definimos las principales entidades del sistema, sus atributos y las relaciones entre ellas, estableciendo claves primarias y foráneas que permitieran garantizar la integridad de los datos y una organización coherente de la información.

Una vez definida la estructura de la base de datos, realizamos el proceso de migración de los datos generados durante la fase anterior. El dataset limpio que anteriormente se almacenaba en archivos CSV fue transformado y volcado a la nueva base de datos, asegurando que la información se mantuviera consistente y correctamente relacionada entre las distintas tablas.

Posteriormente, comenzamos el desarrollo inicial del backend del sistema. En esta etapa se implementaron los primeros endpoints que permiten consultar información almacenada en la base de datos mediante servicios REST. Estos endpoints constituyen el punto de acceso principal a los datos del sistema y sientan las bases para la futura integración con el modelo predictivo.

Finalmente, se desarrolló una interfaz web básica cuyo objetivo principal es mostrar información general del proyecto y verificar el correcto funcionamiento de la comunicación entre frontend y backend. Aunque esta

interfaz es todavía sencilla, permite comprobar que la arquitectura cliente-servidor funciona correctamente y que los distintos componentes del sistema pueden interactuar entre sí.

En conjunto, esta primera fase ha permitido establecer las bases estructurales del sistema TrafiVision, garantizando que la arquitectura formada por base de datos y backend funcione de manera correcta y pueda servir como punto de partida para las siguientes fases del proyecto.

Fase 2: Integración funcional y control de acceso (Fecha de entrega: 26/04)

En la segunda fase el foco se centra en la mejora de la interacción entre frontend y backend, así como en la incorporación de mecanismos básicos de seguridad y usabilidad.

Los objetivos específicos son:

- Aplicar principios básicos de usabilidad en la interfaz web, mejorando la organización visual, la claridad de navegación y la experiencia de usuario.
- Conectar de forma completa el frontend con el backend, permitiendo realizar consultas reales a la base de datos y ejecutar predicciones a través de los endpoints implementados.
- Implementar un sistema de autenticación con login de usuarios.
- Incorporar un sistema de roles (por ejemplo, usuario y administrador) que permita controlar el acceso a determinadas funcionalidades.

En esta fase el sistema dejará de ser únicamente una prueba técnica para convertirse en una aplicación web funcional con control de acceso y separación clara de responsabilidades.

Fase 3: Aplicación completa y preparación para producción (Fecha de entrega: 24/05)

La tercera fase tiene como objetivo consolidar el proyecto y dejarlo preparado para un entorno más cercano al real.

Los objetivos son:

- Completar todas las funcionalidades comprometidas en la propuesta inicial, integrando completamente el modelo predictivo dentro de la arquitectura web.
- Implementar la aplicación web final con todas las vistas necesarias para consulta de datos históricos, realización de predicciones y gestión de usuarios.
- Realizar la paquetización del proyecto, organizando correctamente la estructura de carpetas, dependencias y configuración.
- Preparar la aplicación para su posible despliegue en producción, garantizando modularidad, mantenibilidad y coherencia arquitectónica.

En esta fase final se busca que TrafiVision evolucione desde una solución académica hacia una aplicación web completa, estructurada y técnicamente sólida.

3.2 Objetivos específicos

- En esta primera fase del proyecto, definimos una serie de objetivos específicos orientados a establecer la base técnica sobre la que se apoyará la evolución del sistema TrafiVision. Estos objetivos se centran principalmente en la organización de la arquitectura del sistema, la gestión estructurada de los datos y el desarrollo de los primeros componentes de la aplicación.

- En primer lugar, nos propusimos diseñar un modelo de base de datos relacional capaz de representar de forma estructurada la información generada durante la fase anterior del proyecto. Para ello definimos las principales entidades del sistema, sus atributos y las relaciones existentes entre ellas, garantizando que la estructura de datos reflejara correctamente la lógica del funcionamiento de TrafiVision.
- A partir de este diseño conceptual, implementamos la base de datos física utilizando un sistema gestor de bases de datos relacional. En esta etapa se crearon las tablas necesarias, junto con sus correspondientes claves primarias y foráneas, con el objetivo de garantizar la integridad referencial y una organización coherente de la información.
- Otro de los objetivos fundamentales fue realizar la migración de los datos previamente almacenados en archivos CSV hacia la nueva base de datos. Este proceso implicó transformar y volcar el dataset generado en la fase anterior, asegurando que los registros se almacenaran correctamente y mantuvieran la consistencia entre las distintas tablas del sistema.
- Asimismo, nos planteamos desarrollar un backend inicial que permitiera establecer la conexión con la base de datos y ofrecer un punto de acceso estructurado a la información del sistema. Este backend se encarga de gestionar las consultas a la base de datos y de exponer servicios que puedan ser utilizados por otras partes de la aplicación.
- Como parte de este proceso, se implementaron endpoints REST que permiten recuperar datos históricos relacionados con el tráfico y la información meteorológica mediante peticiones HTTP estructuradas. Estos endpoints constituyen el mecanismo principal de comunicación entre los distintos componentes del sistema.
- Además, realizamos distintas pruebas de consulta y validación con el objetivo de comprobar el correcto funcionamiento de la comunicación entre el backend y la base de datos, asegurando que los datos se recuperan de forma correcta y consistente.

- Por otro lado, se planteó el desarrollo de una interfaz web simple que permitiera mostrar información básica del proyecto y verificar que la conexión con los endpoints implementados funcionaba correctamente.
- Finalmente, otro de los objetivos de esta fase fue organizar la estructura del proyecto siguiendo una arquitectura modular que permita separar claramente los componentes relacionados con la base de datos, el backend y el frontend, facilitando así la ampliación y evolución futura del sistema.

Capítulo 4. DESARROLLO DEL PROYECTO

4.1 Planificación del proyecto

El desarrollo del proyecto TrafiVision en esta nueva fase se ha organizado siguiendo una planificación estructurada por etapas, alineada con los hitos establecidos en el cronograma académico. La planificación combina tareas de análisis, diseño e implementación, asegurando una evolución progresiva desde la arquitectura conceptual hasta una primera versión funcional del sistema.

En la fase actual (Fase 1), las actividades realizadas y planificadas se centran en la definición de la arquitectura base del sistema y en la implementación de sus componentes fundamentales.

Actividades realizadas

1. Análisis de la arquitectura previa

Se realizó un estudio del sistema desarrollado en la fase anterior (PCDI), identificando sus limitaciones estructurales, especialmente en relación con el almacenamiento en archivos CSV, la ausencia de base de datos relacional y la falta de separación entre frontend y backend.

Esfuerzo estimado: 6–8 horas.

2. Diseño del modelo de datos

Se llevó a cabo el diseño conceptual de la base de datos, definiendo entidades principales (cámaras, capturas, datos meteorológicos, detecciones, tráfico), sus atributos y las relaciones entre ellas.

Esfuerzo estimado: 8–10 horas.

3. Diseño lógico y físico de la base de datos

Se transformó el modelo conceptual en un esquema relacional implementable, definiendo claves primarias, claves foráneas y restricciones de integridad.

Esfuerzo estimado: 6–8 horas.

4. Implementación de la base de datos y creación de tablas

Se implementó la base de datos en el sistema gestor seleccionado y se crearon las tablas correspondientes.

Esfuerzo estimado: 6–8 horas.

5. Migración y volcado de datos

Se desarrollaron scripts para transformar y volcar los datos previamente almacenados en CSV hacia la nueva base de datos, verificando su consistencia.

Esfuerzo estimado: 8–10 horas.

6. Desarrollo inicial del backend

Se inició la implementación del backend, estableciendo la conexión con la base de datos y definiendo los primeros endpoints de consulta.

Esfuerzo estimado: 10–12 horas.

7. Implementación de endpoints básicos

Se desarrollaron endpoints REST para consultar información histórica y comprobar el correcto funcionamiento de la arquitectura cliente-servidor.

Esfuerzo estimado: 6–8 horas.

8. Desarrollo de una interfaz web simple

Se creó una primera interfaz web básica que permite visualizar información del proyecto y realizar pruebas de conexión con el backend.

Esfuerzo estimado: 6–8 horas

9. Ampliación del backend y lógica de negocio

Se desarrollaron nuevos endpoints para gestionar funcionalidades completas del sistema, incluyendo predicción de tráfico, gestión de cámaras, control de scraping, actualización de datos meteorológicos y administración de usuarios.

Esfuerzo estimado: 10–12 horas.

10. Integración de modelos de Machine Learning en el backend

Se integraron modelos previamente entrenados en formato `.joblib`, permitiendo realizar predicciones en tiempo real desde el backend a partir de datos introducidos por el usuario.

Esfuerzo estimado: 6–8 horas.

11. Desarrollo de la interfaz web completa

Se evolucionó la interfaz inicial hacia una aplicación web completa en React, incorporando múltiples vistas (dashboard, histórico, predicción, cámaras, administración) y conectándola con el backend mediante peticiones HTTP.

Esfuerzo estimado: 12–14 horas.

12. Visualización de datos históricos y gráficos

Se implementaron componentes para mostrar datos históricos almacenados en la base de datos, incluyendo tablas y gráficos interactivos que permiten analizar tendencias de tráfico.

Esfuerzo estimado: 8–10 horas.

13. Implementación de scraping automatizado desde la web

Se integró el sistema de scraping dentro del backend, permitiendo su ejecución desde la interfaz web, tanto de forma manual como continua en intervalos de tiempo definidos.

Esfuerzo estimado: 6–8 horas.

14. Integración del procesamiento de imágenes (YOLO)

Se conectó el procesamiento de imágenes mediante YOLO al flujo de datos del sistema, permitiendo analizar automáticamente las imágenes capturadas y generar métricas de tráfico.

Esfuerzo estimado: 8–10 horas.

15. Desarrollo del pipeline de datos (ETL)

Se diseñó el flujo de procesamiento completo de datos, incluyendo la combinación de datos de cámaras, resultados de YOLO y datos meteorológicos para generar datasets finales preparados para análisis y modelado.

Esfuerzo estimado: 10–12 horas.

16. Integración del entrenamiento de modelos desde la interfaz web

Se habilitó la ejecución del entrenamiento de modelos directamente desde la aplicación web, conectando el frontend con scripts de entrenamiento en el backend.

Esfuerzo estimado: 6–8 horas.

17. Integración completa del sistema

Se logró la integración de todos los componentes (scraping, procesamiento, base de datos, backend y frontend), obteniendo un sistema funcional de análisis y predicción de tráfico con arquitectura cliente-servidor.

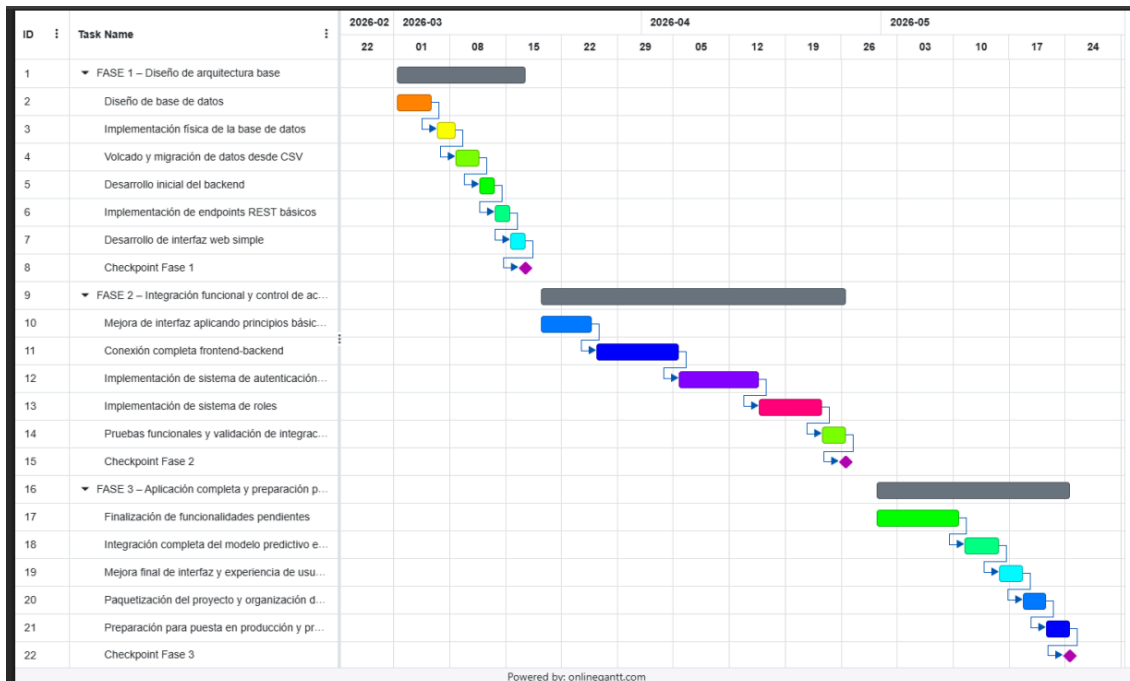
Esfuerzo estimado: 8–10 horas.

Organización temporal

La planificación de esta fase se ha estructurado en un periodo aproximado de cuatro semanas, distribuyendo las tareas de forma progresiva:

- Semana 1: análisis de la arquitectura previa y diseño conceptual de la base de datos.
- Semana 2: diseño lógico e implementación de la base de datos.
- Semana 3: migración de datos y desarrollo inicial del backend.
- Semana 4: implementación de endpoints y desarrollo de interfaz básica.

Esta planificación permite asegurar que cada componente se construya sobre una base previamente validada, reduciendo errores de integración en fases posteriores.



La solución propuesta en el proyecto TrafiVision se basa en la integración de técnicas de ingeniería de datos, aprendizaje automático y desarrollo de software para construir un sistema capaz de analizar información relacionada con el tráfico urbano y generar predicciones a partir de datos históricos.

En la fase inicial del proyecto se desarrolló un sistema orientado principalmente a la recopilación y procesamiento de datos. Este sistema permitía automatizar la obtención de imágenes de cámaras de tráfico públicas, integrar datos meteorológicos procedentes de una API externa y aplicar técnicas de visión por computador para detectar vehículos en las imágenes. A partir de esta información se generaba un dataset estructurado que posteriormente se utilizaba para entrenar distintos modelos de aprendizaje automático capaces de estimar el nivel de tráfico.

En la fase actual del proyecto, el enfoque se ha centrado en mejorar la organización del sistema y su arquitectura. En concreto, se ha llevado a cabo la migración del almacenamiento de datos hacia una base de datos relacional, sustituyendo el uso previo de archivos CSV. Además, se ha implementado un módulo de conexión que centraliza las consultas y operaciones sobre los datos, permitiendo una gestión más eficiente y estructurada de la información.

Desde el punto de vista metodológico, el desarrollo del proyecto se ha basado en una estrategia incremental. En primer lugar, se analizó la arquitectura existente para identificar sus principales limitaciones, especialmente en relación con el uso de archivos CSV como mecanismo de almacenamiento y la falta de modularidad. A partir de este análisis, se diseñó una nueva arquitectura basada en el uso de una base de datos relacional, lo que ha permitido mejorar la escalabilidad y mantenibilidad del sistema.

Actualmente, el sistema se apoya en una base de datos que almacena información relacionada con las cámaras de tráfico, las capturas realizadas, los datos meteorológicos, las detecciones de vehículos y los registros agregados de tráfico. Esta estructura permite organizar los datos de forma coherente y facilita su consulta para posteriores análisis o predicciones.

En cuanto al desarrollo del software, se ha utilizado el lenguaje de programación Python como tecnología principal del proyecto. Python se emplea tanto para el acceso a la base de datos como para el entrenamiento y ejecución de los modelos de aprendizaje automático. Para la conexión con la base de datos se ha desarrollado un módulo específico que centraliza las operaciones de consulta y recuperación de datos, permitiendo reutilizar estas funcionalidades en distintos componentes del sistema.

El entrenamiento de los modelos predictivos se ha realizado utilizando la librería Scikit-learn, que proporciona diferentes algoritmos de aprendizaje supervisado. En el proyecto se han entrenado y evaluado varios modelos, entre los que se incluyen Random Forest, Árboles de decisión, K-Nearest Neighbors y Regresión Logística. Estos modelos se entrenan utilizando los datos históricos almacenados en la base de datos y posteriormente se guardan mediante archivos serializados que permiten reutilizarlos durante la ejecución de la aplicación.

Por otro lado, se ha desarrollado una interfaz de usuario que permite interactuar con el sistema y acceder a sus principales funcionalidades. Inicialmente, esta interfaz se implementó como una aplicación de escritorio utilizando la librería CustomTkinter. Posteriormente, en la fase actual, se ha comenzado el desarrollo de una interfaz web que se comunica con el backend mediante una API REST, permitiendo la gestión de procesos como

scraping, actualización de datos, visualización del histórico y entrenamiento de modelos.

En este contexto, se ha implementado un backend basado en FastAPI que permite exponer endpoints para la consulta de datos, la ejecución de procesos ETL y el entrenamiento de modelos. Esta arquitectura cliente-servidor mejora la separación de responsabilidades y facilita futuras ampliaciones del sistema.

Actualmente, algunas funcionalidades avanzadas, como la automatización completa del flujo ETL (scraping → procesamiento → almacenamiento en base de datos) y la integración total del entrenamiento de modelos desde la interfaz web, se encuentran en proceso de consolidación, habiéndose desarrollado ya gran parte de los componentes necesarios.

Como líneas futuras de desarrollo, se plantea completar la integración total del sistema, mejorar la automatización de los procesos de ingesta de datos, optimizar el rendimiento de los modelos predictivos e incorporar nuevas funcionalidades en la interfaz web, como visualizaciones avanzadas y paneles de control en tiempo real.

En conjunto, la solución desarrollada combina técnicas de análisis de datos, aprendizaje automático y desarrollo de software para construir un sistema funcional que permite analizar información de tráfico urbano y generar predicciones a partir de ella, sentando una base sólida para futuras ampliaciones del proyecto.

Recursos requeridos

Para el desarrollo del proyecto TrafiVision se han utilizado distintos recursos técnicos y herramientas de software que han permitido llevar a cabo las tareas de recopilación de datos, procesamiento de información, entrenamiento de modelos predictivos y desarrollo de la aplicación. A continuación, se

enumeran los principales recursos empleados durante la ejecución del proyecto.

Recursos hardware

- Ordenadores personales utilizados por los integrantes del equipo para el desarrollo del proyecto.
- Conexión a Internet para la descarga de datos, acceso a APIs externas y uso de repositorios de código.

Recursos software

- Python 3 como lenguaje principal de programación.
- Visual Studio Code como entorno de desarrollo integrado (IDE).
- Git y GitHub para el control de versiones y la gestión del repositorio del proyecto.

Librerías y frameworks utilizados

- pandas para el procesamiento y manipulación de datos.
- requests para realizar peticiones a APIs externas.
- OpenCV para el procesamiento básico de imágenes.
- Ultralytics YOLOv8 para la detección de vehículos en imágenes de tráfico.
- PyTorch como base para la ejecución del modelo de visión por computador.
- scikit-learn para el entrenamiento de modelos de aprendizaje automático.
- joblib para la serialización y almacenamiento de modelos entrenados.
- CustomTkinter para el desarrollo inicial de la interfaz gráfica de escritorio.
- FastAPI para el desarrollo del backend y la creación de endpoints REST.
- React (con Vite) para el desarrollo de la interfaz web del sistema.
- pymysql para la conexión entre Python y la base de datos relacional.

Base de datos

- Sistema gestor de base de datos relacional MariaDB, utilizado para el almacenamiento estructurado de la información generada por el sistema, sustituyendo el uso previo de archivos CSV.

Fuentes de datos externas

- Cámaras públicas de tráfico del Ayuntamiento de Madrid.
- API meteorológica Open-Meteo para la obtención de datos climáticos.

Estos recursos han permitido desarrollar todas las tareas del proyecto, desde la recopilación y tratamiento de datos hasta la implementación de la arquitectura completa del sistema, incluyendo backend, base de datos, modelos de aprendizaje automático e interfaz de usuario.

4.2 Presupuesto

El presupuesto del proyecto TrafiVision se ha estimado considerando tanto el tiempo de dedicación del equipo como los recursos técnicos utilizados durante su desarrollo. Aunque se trata de un proyecto académico, se ha realizado una valoración económica aproximada para simular un entorno profesional.

El equipo está compuesto por tres integrantes, con una dedicación media estimada de cuatro horas semanales por persona durante un periodo aproximado de doce semanas. Esto supone un total de 144 horas de trabajo conjunto.

Se ha estimado un coste medio de 15 € por hora, correspondiente a un perfil junior en el ámbito del desarrollo de software.

A continuación, se presenta el resumen económico del proyecto:

Resumen de presupuesto

Tipo de coste	Valor	Comentarios
Horas de trabajo en el proyecto	2.160 €	144 horas totales (3 personas × 48 horas) × 15 €/hora
Equipo técnico utilizado	3.000 €	3 ordenadores personales (valor aproximado 1.000 € cada uno, aunque no se han adquirido específicamente para el proyecto)
Software utilizado	0 €	Uso de software libre y herramientas sin coste de licencia (Python, VS Code, gestor de base de datos, librerías open-source)
Estudios e informes	0 €	No se han adquirido informes externos ni documentación de pago
Materiales empleados	0 €	No se han requerido materiales físicos adicionales

Coste total estimado del proyecto: 5.160 €

(2.160 € en horas + 3.000 € en equipos)

4.3 Viabilidad

Este apartado es opcional, pero es interesante que incluyas un breve análisis de viabilidad económica del proyecto (relación coste / beneficio), y un análisis de sostenibilidad a futuro del resultado de tu proyecto.

4.4 Resultados del proyecto

Conforme a los objetivos específicos, describe los resultados finales obtenidos.

Para proyectos con un enfoque de desarrollo de producto, debes incluir el resultado de tu plan de pruebas.

Puedes añadir una descripción de cambios durante el proyecto respecto a los objetivos iniciales, y comentarios que quieras incluir de cómo has desarrollado las distintas actividades.

Capítulo 5. TECNOLOGIAS USADAS

El proyecto TrafiVision ha evolucionado desde una solución orientada a la ingeniería de datos y modelado predictivo en entorno local (PCDI) hacia una arquitectura web estructurada (PCII).

En este capítulo se describen las tecnologías utilizadas hasta el momento y las nuevas herramientas incorporadas en la fase actual del proyecto.

5.1 Tecnologías utilizadas en la fase anterior (Proyecto de Computación I)

Durante la primera fase del proyecto, el desarrollo estuvo centrado en la automatización del proceso ETL, el análisis de imágenes mediante visión por computador y el modelado predictivo.

Lenguaje principal

Python 3.9

Se utilizó como lenguaje base para todo el desarrollo del sistema, incluyendo extracción de datos, transformación, análisis de imágenes y modelado.

Librerías de extracción y procesamiento de datos

requests

Para realizar peticiones HTTP a las cámaras públicas y a la API meteorológica.

pandas

Para limpieza, transformación, integración y exportación de datos en formato CSV.

datetime y schedule

Para la gestión temporal y automatización periódica de capturas.

Visión por computador

Ultralytics – YOLOv8

Modelo de detección de objetos empleado para identificar vehículos en imágenes de tráfico.

Se utilizó la versión ligera *yolov8n* por su eficiencia en ejecución local.

OpenCV

Para procesamiento básico de imágenes.

PyTorch (torch y torchvision)

Como base para la ejecución del modelo YOLOv8.

Modelado predictivo**Scikit-learn**

Para entrenamiento de modelos supervisados (Árboles de decisión, Random Forest, KNN, Regresión Logística).

Altair AI Studio

Para experimentación visual, validación cruzada y comparación de algoritmos.

joblib

Para serialización y reutilización de modelos entrenados.

Entorno y control de versiones**Visual Studio Code**

Como entorno de desarrollo principal.

Git y GitHub

Para control de versiones y organización del proyecto.

5.2 Tecnologías incorporadas en la fase actual

En la fase actual del proyecto, TrafiVision evoluciona hacia una arquitectura cliente-servidor estructurada, incorporando tecnologías orientadas a mejorar la persistencia de datos, la organización del sistema y la comunicación entre los distintos componentes de la aplicación.

Esta evolución permite superar las limitaciones del prototipo inicial, basado en scripts locales y almacenamiento en archivos CSV, dando lugar a una solución más escalable, mantenible y alineada con estándares profesionales de desarrollo software.

Base de datos

Sistema Gestor de Base de Datos Relacional (MariaDB / MySQL)

En esta fase se ha implementado una base de datos relacional que sustituye el sistema previo basado en archivos CSV. Este cambio permite almacenar la información de forma estructurada, garantizando la integridad referencial, la consistencia de los datos y una mayor eficiencia en las operaciones de consulta.

La base de datos centraliza la información del sistema, incluyendo cámaras, capturas, datos meteorológicos, detecciones y registros de tráfico, facilitando su explotación para análisis y predicción.

SQL (Structured Query Language)

El lenguaje SQL se emplea para la definición del esquema de la base de datos, incluyendo la creación de tablas, claves primarias y foráneas, así como las relaciones entre entidades. Asimismo, se utiliza para la realización de consultas que permiten recuperar, filtrar y gestionar los datos de manera eficiente.

Backend

Framework backend en Python (FastAPI)

Para el desarrollo del backend se ha utilizado un framework web ligero basado en Python, concretamente FastAPI, que permite construir APIs modernas, eficientes y fácilmente escalables.

El backend actúa como capa intermedia entre la base de datos y la interfaz de usuario, centralizando la lógica de negocio del sistema. A través de esta capa se implementan funcionalidades clave como:

- Definición de endpoints REST para el acceso a los datos.

- Gestión de peticiones HTTP (GET, POST).
- Conexión y operaciones sobre la base de datos.
- Ejecución de procesos como scraping, actualización de datos y predicciones.
- Exposición de servicios al frontend mediante respuestas en formato JSON.

En esta fase se han desarrollado endpoints tanto de consulta como de ejecución de procesos, permitiendo una interacción completa con el sistema desde la interfaz web.

Arquitectura REST

El sistema adopta una arquitectura basada en servicios REST, lo que permite una comunicación estructurada entre el frontend y el backend mediante peticiones HTTP y respuestas en formato JSON.

Este enfoque proporciona múltiples ventajas:

- Separación clara entre frontend y backend.
- Independencia entre los distintos componentes del sistema.
- Mayor facilidad de mantenimiento y evolución.
- Escalabilidad para incorporar nuevos servicios o clientes en el futuro.

Gracias a esta arquitectura, TrafiVision se consolida como una plataforma modular preparada para crecer hacia soluciones más complejas.

Frontend

Aplicación web (React + Vite)

En la fase actual se ha desarrollado una interfaz web moderna que sustituye y amplía la funcionalidad de la interfaz inicial. Esta aplicación se comunica con el backend a través de la API REST, permitiendo al usuario interactuar con el sistema de forma dinámica.

La interfaz web permite:

- Visualizar datos históricos almacenados en la base de datos.
- Consultar información de cámaras de tráfico.

- Ejecutar procesos como scraping y actualización de datos.
- Realizar predicciones mediante modelos de Machine Learning.
- Gestionar funcionalidades administrativas del sistema.

El uso de React facilita la creación de una interfaz modular, reutilizable y escalable, mientras que Vite permite un entorno de desarrollo rápido y eficiente.

Conclusión

La incorporación de estas tecnologías ha permitido transformar TrafiVision en un sistema estructurado basado en una arquitectura cliente-servidor moderna. Esta nueva base tecnológica mejora significativamente la organización del proyecto, la gestión de los datos y la capacidad de evolución futura, sentando las bases para la incorporación de nuevas funcionalidades y mejoras en fases posteriores.

5.3 Evolución tecnológica del proyecto

La evolución tecnológica del proyecto TrafiVision puede entenderse como un proceso progresivo de maduración del sistema, pasando de un prototipo orientado al análisis de datos a una arquitectura estructurada alineada con entornos reales de desarrollo software.

En una primera etapa, el desarrollo se centró en la construcción de un sistema basado en scripts de Python cuyo objetivo principal era la recopilación, procesamiento e integración de datos. En esta fase se implementaron mecanismos automatizados para la obtención de imágenes de cámaras de tráfico, la integración de información meteorológica a través de APIs externas y la aplicación de técnicas de visión por computador para la detección de vehículos mediante modelos basados en YOLO. A partir de estos datos se generaron datasets estructurados que permitieron entrenar distintos modelos de aprendizaje automático orientados a la predicción del nivel de tráfico.

Posteriormente, el proyecto evolucionó hacia una fase centrada en la mejora de la arquitectura del sistema. En este punto se abordó la necesidad de estructurar la solución desde un enfoque más robusto y escalable. Para ello, se sustituyó el almacenamiento basado en archivos CSV por una base de datos relacional, lo que permitió gestionar la información de forma persistente, estructurada y eficiente. Paralelamente, se desarrolló un backend basado en FastAPI que actúa como capa intermedia entre la base de datos y los clientes, permitiendo centralizar la lógica de negocio y exponer los datos mediante endpoints REST.

De forma complementaria, se ha incorporado una interfaz web que permite interactuar con el sistema a través de una arquitectura cliente-servidor. Esta interfaz facilita la visualización de datos, la ejecución de procesos y la interacción con los modelos predictivos, consolidando la transición hacia una aplicación más completa y orientada al usuario.

Actualmente, el sistema integra los distintos componentes desarrollados —scraping de datos, procesamiento de imágenes, base de datos, backend y frontend— en una arquitectura unificada que permite gestionar el flujo completo de información. No obstante, algunos procesos, como la automatización total del pipeline de datos o la integración completa del entrenamiento de modelos desde la interfaz web, se encuentran en proceso de consolidación.

Como línea de evolución futura, se prevé reforzar la integración entre los distintos módulos del sistema, mejorar la automatización de los procesos de ingesta y procesamiento de datos, incorporar mecanismos de autenticación y gestión de usuarios, y preparar la aplicación para su posible despliegue en entornos más cercanos a producción.

En conjunto, esta evolución tecnológica refleja la transición desde una solución basada en ejecución local y scripts independientes hacia una aplicación modular, escalable y basada en una arquitectura cliente-servidor, más alineada con las prácticas profesionales actuales en el desarrollo de software.

Capítulo 6. ANALISIS Y DISEÑO DE LA SOLUCIÓN

En este capítulo se describe el análisis realizado para abordar el problema identificado en el proyecto, así como el diseño de la solución propuesta. El objetivo de esta sección es explicar cómo se ha estructurado el sistema TrafiVision para evolucionar desde el prototipo desarrollado en la fase anterior hacia una solución más organizada y preparada para su ampliación en futuras fases del proyecto.

Como se ha descrito en capítulos anteriores, el sistema inicial permitía recopilar datos de tráfico y meteorología, generar un dataset estructurado y entrenar modelos de aprendizaje automático capaces de estimar el nivel de tráfico urbano. Sin embargo, la arquitectura utilizada en esa fase estaba orientada principalmente a la experimentación local, lo que limitaba la escalabilidad del sistema y su posible evolución hacia una aplicación más completa.

A partir de este análisis, se identificó la necesidad de rediseñar la arquitectura del sistema con el objetivo de mejorar la gestión de los datos, separar claramente los distintos componentes del proyecto y facilitar la incorporación de nuevas funcionalidades en fases posteriores.

La solución propuesta se basa en la organización del sistema en varios componentes principales que interactúan entre sí. En primer lugar, se incorpora una base de datos relacional encargada de almacenar de forma estructurada toda la información generada por el sistema, incluyendo datos de cámaras, capturas, información meteorológica, detecciones de vehículos y registros de tráfico.

En segundo lugar, se plantea la existencia de una capa de backend que actúa como intermediaria entre la base de datos y las aplicaciones que utilizan el sistema. Esta capa se encarga de gestionar las consultas a la base de datos, procesar la información necesaria y exponer servicios que permitan acceder a los datos o ejecutar predicciones utilizando los modelos entrenados.

Por otro lado, el sistema incorpora una interfaz de usuario que permite interactuar con las funcionalidades del proyecto. A través de esta interfaz es

posible consultar información almacenada en la base de datos, visualizar registros históricos y realizar predicciones de tráfico a partir de distintos parámetros de entrada.

Desde el punto de vista tecnológico, la solución se basa principalmente en el uso del lenguaje de programación Python, que se utiliza tanto para el acceso a la base de datos como para la implementación de los modelos de aprendizaje automático y el desarrollo de la aplicación. Asimismo, se utilizan tecnologías de bases de datos relacionales para gestionar la información de forma estructurada y facilitar su consulta.

El diseño del sistema sigue una arquitectura modular en la que cada componente cumple una función específica dentro del proyecto. Esta separación de responsabilidades permite mejorar la organización del código, facilita el mantenimiento del sistema y permite que los distintos módulos puedan evolucionar de forma independiente.

En conjunto, el análisis y diseño realizados establecen las bases para la implementación del sistema TrafiVision en su fase actual, permitiendo transformar el prototipo inicial en una solución más estructurada y preparada para futuras ampliaciones.

Capítulo 7. IMPLEMENTACIÓN

En este capítulo se describe cómo se ha llevado a cabo la implementación de la solución propuesta en el capítulo anterior. El objetivo no es detallar paso a paso el desarrollo técnico del sistema, sino explicar las principales etapas que se han seguido durante el proceso de implementación y las decisiones técnicas adoptadas en cada una de ellas.

Durante esta fase del proyecto trabajamos en la construcción de los distintos componentes que forman el sistema TrafiVision, incluyendo la base de datos, el acceso a los datos desde Python, el entrenamiento de los modelos de aprendizaje automático y el desarrollo de una aplicación que permite interactuar con el sistema.

7.1 Implementación de la base de datos

Uno de los primeros pasos de la implementación consistió en sustituir el sistema de almacenamiento basado en archivos CSV por una base de datos relacional. Esta decisión se tomó con el objetivo de mejorar la organización de los datos, garantizar su persistencia y facilitar su consulta desde distintas partes del sistema.

Para ello se diseñó una estructura de base de datos compuesta por varias tablas que representan las principales entidades del proyecto. Entre estas tablas se incluyen la información de las cámaras de tráfico, las capturas realizadas por el sistema, los datos meteorológicos obtenidos de la API externa, las detecciones de vehículos realizadas mediante visión por computador y los registros agregados de tráfico.

La base de datos se implementó utilizando un sistema gestor relacional y se definieron las correspondientes claves primarias y foráneas para mantener la integridad de la información. Esta estructura permite organizar los datos de forma coherente y facilita la realización de consultas complejas sobre el dataset generado por el sistema.

7.2 Migración de datos desde archivos CSV

Una vez creada la estructura de la base de datos, el siguiente paso consistió en migrar los datos generados durante la fase anterior del proyecto. En la etapa inicial, toda la información se almacenaba en archivos CSV generados durante el proceso ETL.

Para realizar esta migración desarrollamos scripts en Python que permitían leer los archivos CSV existentes, transformar los datos cuando era necesario y volcarlos a las tablas correspondientes de la base de datos. Durante este proceso se realizaron distintas comprobaciones para asegurar que los registros se almacenaban correctamente y que se mantenía la coherencia entre las distintas entidades.

Este proceso permitió trasladar el dataset previamente generado a un sistema de almacenamiento más robusto y preparado para futuras ampliaciones del proyecto.

7.3 Implementación del acceso a la base de datos

Con el objetivo de facilitar la interacción entre la aplicación y la base de datos, se desarrolló un módulo específico encargado de gestionar la conexión y las consultas a la base de datos.

Este módulo centraliza las operaciones relacionadas con el acceso a los datos, permitiendo realizar consultas para recuperar información histórica, obtener estadísticas del sistema o generar los datasets necesarios para el entrenamiento de los modelos predictivos.

La centralización de estas funcionalidades en un único módulo permite mantener el código más organizado y facilita su reutilización en distintos componentes del proyecto.

7.4 Entrenamiento de los modelos de aprendizaje automático

Otra de las etapas importantes de la implementación fue el entrenamiento de los modelos de aprendizaje automático utilizados para predecir el nivel de tráfico.

Para ello se utilizaron los datos históricos generados por el sistema y almacenados en la base de datos. A partir de estos datos se prepararon los

conjuntos de entrenamiento necesarios para aplicar distintos algoritmos de aprendizaje supervisado.

Entre los modelos implementados se encuentran algoritmos como Random Forest, Árboles de decisión, K-Nearest Neighbors y Regresión Logística. Estos modelos fueron entrenados utilizando la librería Scikit-learn y posteriormente evaluados para comprobar su capacidad de predicción.

Una vez entrenados, los modelos se almacenaron mediante archivos serializados que permiten reutilizarlos durante la ejecución de la aplicación sin necesidad de volver a entrenarlos cada vez.

7.5 Desarrollo de la aplicación de escritorio

Para facilitar la interacción con el sistema se desarrolló una aplicación de escritorio que permite acceder a las principales funcionalidades del proyecto.

Esta aplicación fue implementada utilizando la librería CustomTkinter, que permite crear interfaces gráficas en Python con un diseño más moderno que las bibliotecas tradicionales.

A través de esta aplicación es posible consultar los registros almacenados en la base de datos, explorar información histórica, visualizar estadísticas generales del sistema y realizar predicciones de tráfico introduciendo distintos parámetros de entrada.

La aplicación se organiza en distintas ventanas o módulos que permiten acceder a las diferentes funcionalidades del sistema, facilitando así su uso por parte del usuario.

7.6 Problemas encontrados durante la implementación

Durante el proceso de implementación surgieron diversas dificultades técnicas que fue necesario resolver para poder completar el desarrollo del sistema.

Uno de los principales retos estuvo relacionado con el proceso de migración de los datos desde los archivos CSV hacia la base de datos relacional. Fue necesario adaptar algunos formatos de datos y revisar la correspondencia

entre las distintas entidades para asegurar que la información se almacenara correctamente.

También surgieron algunos problemas relacionados con la conexión entre la aplicación y la base de datos, especialmente durante las primeras pruebas de consulta. Estos problemas se resolvieron revisando la configuración de la conexión y reorganizando el módulo encargado del acceso a los datos.

Otro aspecto que requirió especial atención fue la integración de los modelos de aprendizaje automático dentro de la aplicación. Fue necesario asegurar que los modelos entrenados pudieran cargarse correctamente y utilizarse para generar predicciones a partir de los datos introducidos por el usuario.

A pesar de estas dificultades, todas ellas pudieron resolverse durante el desarrollo del proyecto, permitiendo obtener una versión funcional del sistema TrafiVision en su estado actual.

Capítulo 8. CONCLUSIONES

EL NUMERO DE ESTE CAPÍTULO VA A DEPENDER DEL CUERPO DEL TRABAJO QUE SERÁ EL ANALISIS E IMPLEMENTACIÓN.

8.1 Conclusiones del trabajo

Breve descripción objetiva del resultado en relación con el objetivo general de tu proyecto.

8.2 Puntos débiles del trabajo.

Breve descripción de aquello que ya se ha visto que puede ser un problema. Por ejemplo, en la actualización de la red que por problemas de presupuesto no se ha hecho el backbone con fibra óptica y se mantiene la implementación existente.

8.3 Conclusiones personales

Describe tus impresiones y experiencia personal durante el desarrollo del proyecto, o destacar la importancia que tiene el tema para ti, lo que has aprendido, o la trascendencia que ha tenido para ti o para otros este proyecto.

8.4 Futuras mejoras

En este apartado las líneas con las que se podría continuar tu trabajo. Indica todo lo que has anotado como futuro trabajos durante el desarrollo del proyecto, o aquellos aspectos que estaban fuera del alcance, pero que son interesantes para desarrollar a futuro el valor de los resultados de tu trabajo.

Capítulo 9. GLOSARIO

(OPCIONAL)

LISTA DE TÉRMINOS O ACRÓNIMOS QUE SE CONSIDERA QUE NO SON HABITUALES PARA EL LECTOR Y QUE PUEDEN NECESITAR UNA PEQUEÑA DESCRIPCIÓN.

VAN SIEMPRE EN ORDEN ALFABÉTICO.

Capítulo 10. BIBLOGRAFIA

En este apartado figurará el conjunto de libros, revistas u otros textos que el autor considere de interés para justificar las soluciones adoptadas en el Proyecto. **Cita todas las fuentes** que has utilizado como consulta para elaborar el trabajo.

Sigue el estilo de cita que te indiquen las normas de estilo y respétalo a lo largo de todo el proyecto. Recuerda que has de citar todas las fuentes que hayas usado. Los estilos de cita más comunes son:

- ISO
- IEEE
- APPA
- Etc.

En ingeniería se suele usar el ISO o el IEEE.

Si puedes usar un gestor de citas bibliográficas te será más fácil. Si no tendrás que recurrir a las páginas web de las bibliotecas para saber cómo citar adecuadamente. Por ejemplo, lo encuentras en:

<http://biblioteca.uem.es/es/aprendizaje-y-formacion/citas-bibliograficas-documentos>

Referencias usadas en este manual de estilo:

AENOR. 2010. AEN/CTN 157 - PROYECTOS. *Normas y Publicaciones*. [En línea] 2010. [Citado el: 25 de abril de 2013.]

<http://www.aenor.es/aenor/normas/ctn/fichactn.asp?codigonorm=AEN/CTN%20157>.

Miró Julià, José. 2010. Recursos para aprender a escribir. [En línea] 2010.

<http://bioinfo.uib.es/~joemiro/RecEscr/manual.pdf>.

UNE 157001. 2002. Criterios generales para la elaboración de proyectos. *Escuela Universitaria de Ingeniería de Vitoria*. [En línea] 2002. [Citado el: 25 de abril de 2013.]

http://www.coiib.es/coiib/documentos/DocumentosContenidos/Gu%C3%ADa%20de%20elaboraci%C3%B3n%20de%20proyectos/2-Electricidad/5_PNE_157701_Criterios.pdf.

Capítulo 11. ANEXOS

Sirven para incluir documentación complementaria (planos, circuitos, código, ficheros de configuración, especificaciones técnicas y hojas de características, fichas explicativas, resultado de encuestas, reglamentación y normativas requeridas, etc.).

[PÁGINA INTENCIONADAMENTE EN BLANCO]